GLOBALRAIN

Practices for Secure Software Report

Table of Contents

Document Revision History

Version	Date	Author	Comments
1.0	6/17/2026	Haley Candia Perez	

Client

Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Haley Candia Perez

1. Algorithm Cipher

The encryption algorithm selected for Artemis Financial's web application is SHA-256 (Secure Hash Algorithm 256-bit), which is part of the SHA-2 family developed by the National Security Agency (NSA) and published by the National Institute of Standards and Technology (NIST) in 2001.

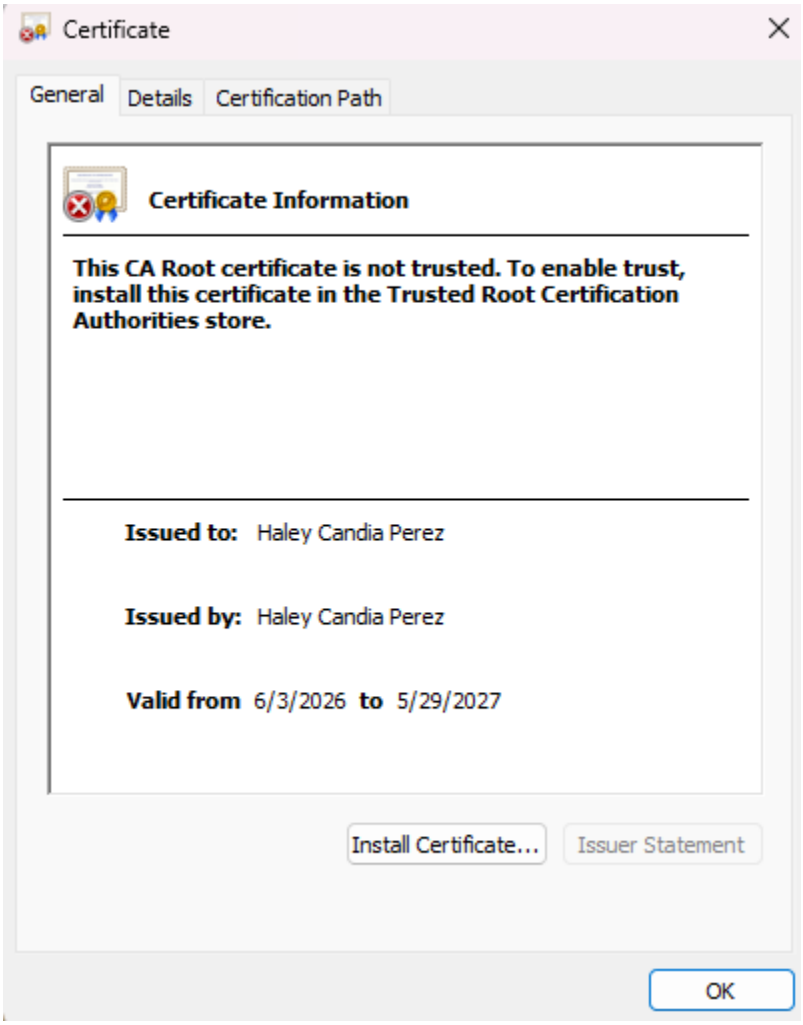
SHA-256 is a cryptographic hash function, meaning it takes an input of any size and produces a fixed length 256-bit (32 byte) output called a hash or digest. It is a one-way function, meaning the original data cannot be reconstructed from the hash, making it ideal for data verification such as the checksum verification required by Artemis Financial.

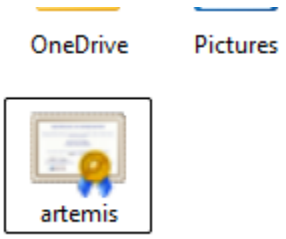
In terms of its key properties, SHA-256 converts input data into a fixed 256-bit digest that uniquely represents the original data. Operating at 256 bits provides a very large number of possible hash values, making collisions extremely unlikely. Unlike symmetric encryption algorithms such as AES, which use a shared secret key, and asymmetric encryption algorithms such as RSA, which use public and private key pairs, SHA-256 is a one-way hashing algorithm that does not use encryption keys. Instead, it relies on bitwise operations, modular addition, and compression functions to generate its output. While SHA-256 itself does not use random numbers, randomness plays an important role in secure communications through TLS certificates and cryptographic key generation, where cryptographically secure random number generators help create unpredictable keys. This distinction makes SHA-256 ideal for integrity verification rather than encryption. SHA-256 is currently considered secure and is widely used in SSL/TLS certificates, digital signatures, password hashing, and blockchain technology.

Historically, encryption algorithms have evolved significantly over time. Early standards such as DES (Data Encryption Standard), introduced in the 1970's, were eventually deemed insecure due to their short key lengths. This led to the development of stronger algorithms such as AES and the SHA family of hash functions. SHA-1, the predecessor to SHA-256, was deprecated by NIST in 2011 after researchers demonstrated its vulnerability to collision attacks. SHA-256, part of the SHA-2 family released in 2001, remains the current standard for cryptographic hashing and is widely trusted by governments, financial institutions, and technology companies worldwide. It continues to be recommended by NIST as a secure hashing algorithm for data integrity verification.

2. Certificate Generation

Insert a screenshot below of the CER file.

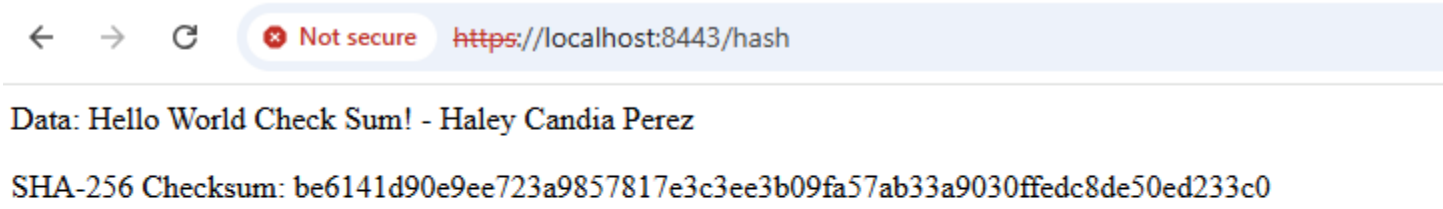




```
PS C:\Users\hthat\Downloads\CS 305 Project Two Code Base\ssl-server_student> keytool -printcert -file "C:\Users\hthat\artemis.cer"
Owner: CN=Haley Candia Perez, OU=SNHU, O=SNHU, L=Brooklyn, ST=NY, C=US
Issuer: CN=Haley Candia Perez, OU=SNHU, O=SNHU, L=Brooklyn, ST=NY, C=US
Serial number: 57c45f6d8673e93e
Valid from: Wed Jun 03 23:36:23 EDT 2026 until: Sat May 29 23:36:23 EDT 2027
Certificate fingerprints:
         SHA1: FE:C4:66:57:35:6C:92:B5:86:51:BA:A7:B3:29:B5:42:E1:05:C2:C3
         SHA256: 97:D1:9C:2E:24:A6:FF:E3:FC:2B:1A:6A:A2:68:57:92:EE:D3:74:6A:F9:53:F0:BA:42:51:57:48:B1:A0:F3:DA
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 17 43 5D 8D 6D C4 8E CA   94 6E 0D F4 1D C9 32 5A  .C].m....n....2Z
0010: 42 74 5C D3                                        Bt\.
]
]

PS C:\Users\hthat\Downloads\CS 305 Project Two Code Base\ssl-server_student>
```

3. Deploy Cipher

Insert a screenshot below of the checksum verification.

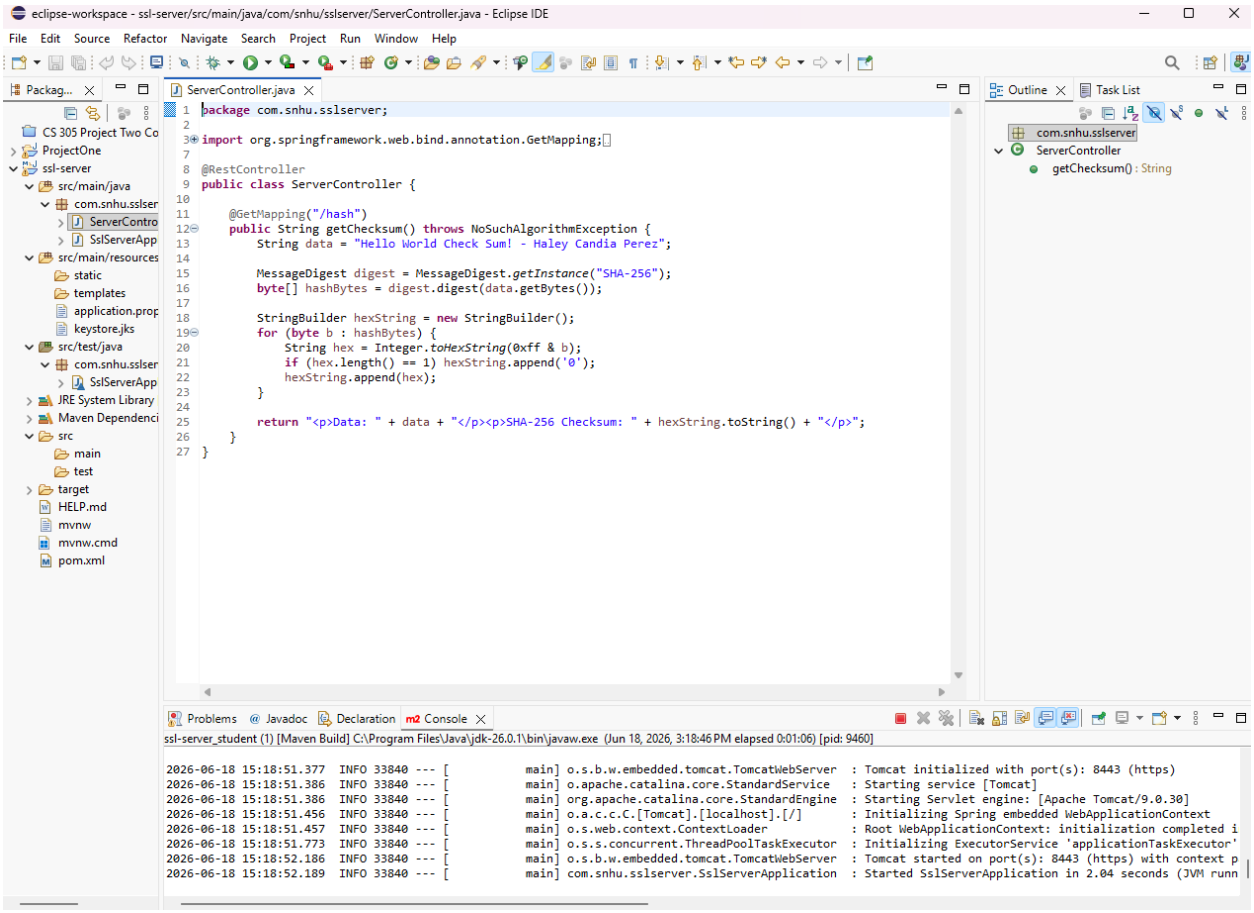


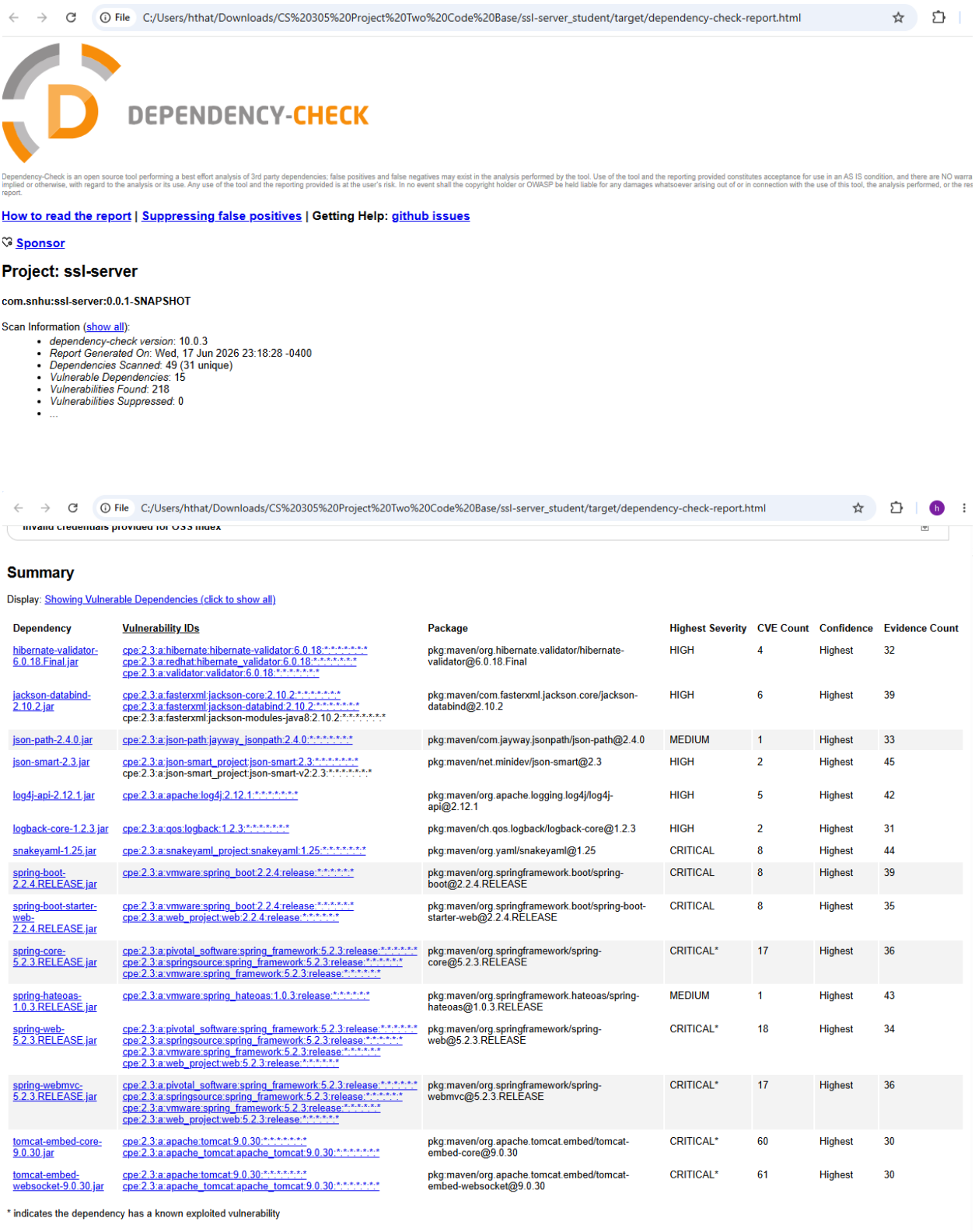
4. Secure Communications

Insert a screenshot below of the web browser that shows a secure webpage.

5. Secondary Testing

Insert screenshots below of the refactored code executed without errors and the dependency-check report.





File C:/Users/hthat/Downloads/CS%20305%20Project%20Two%20Code%20Base/ssl-server_student/target/dependency-check-report.html

DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warra implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the res report.

How to read the report | Suppressing false positives | Getting Help: github issues

Sponsor

Project: ssl-server

com.snhu:ssl-server:0.0.1-SNAPSHOT

Scan Information (show all):

- dependency-check version: 10.0.3
- Report Generated On: Wed, 17 Jun 2026 23:18:28 -0400
- Dependencies Scanned: 49 (31 unique)
- Vulnerable Dependencies: 15
- Vulnerabilities Found: 218
- Vulnerabilities Suppressed: 0
- ...

File C:/Users/hthat/Downloads/CS%20305%20Project%20Two%20Code%20Base/ssl-server_student/target/dependency-check-report.html

Invalid credentials provided for OSS Index

Summary

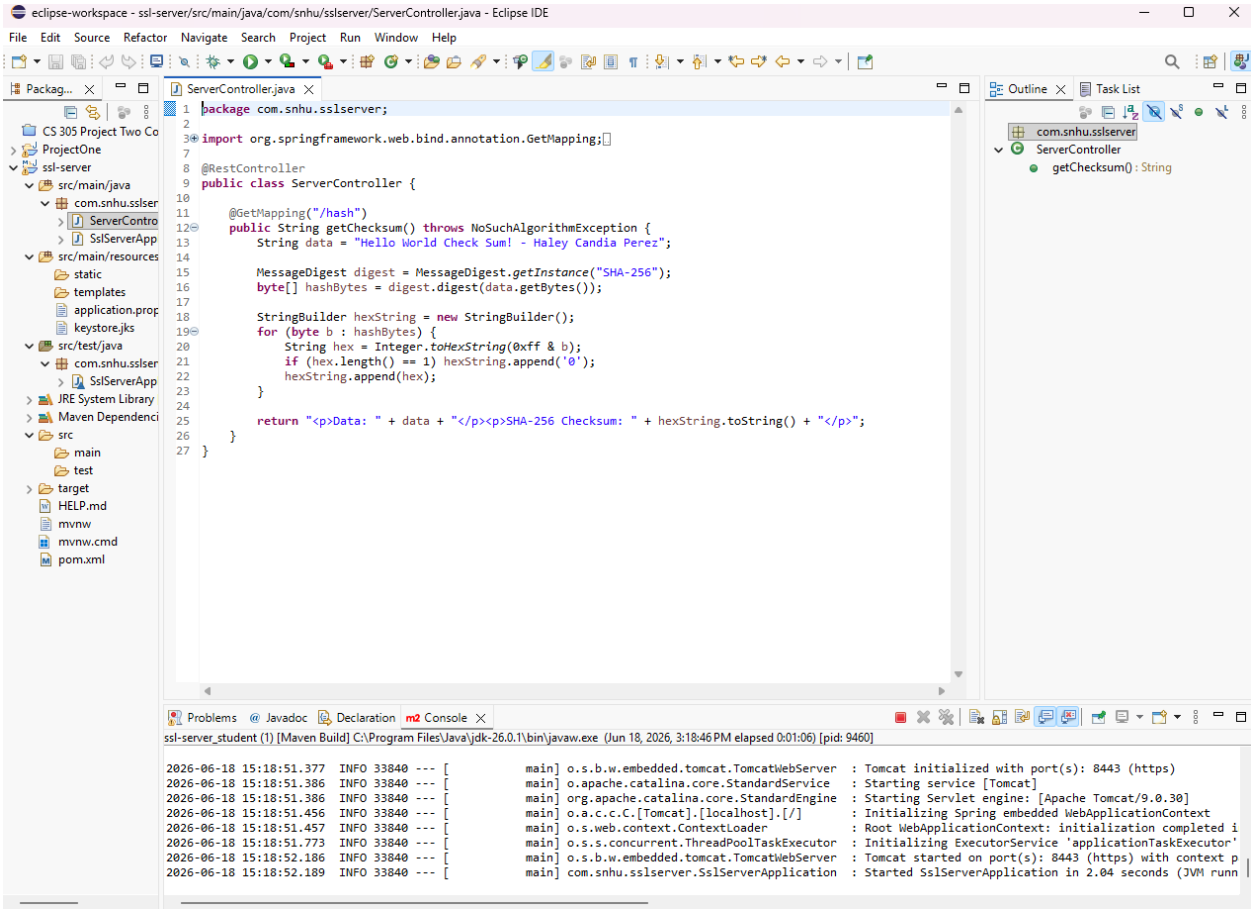
Display: Showing Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:*:*:*:*:* cpe:2.3:a:redhat:hibernate_validator:6.0.18:*:*:*:*:*:*:* cpe:2.3:a:validator:validator:6.0.18:*:*:*:*:*:*:*	pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final	HIGH	4	Highest	32
jackson-databind-2.10.2.jar	cpe:2.3:a:fasterxml:jackson-core:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-databind:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-modules-java8:2.10.2:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.10.2	HIGH	6	Highest	39
json-path-2.4.0.jar	cpe:2.3:a:json-path:jayway_jsonpath:2.4.0:*:*:*:*:*:*:*	pkg:maven/com.jayway.jsonpath/json-path@2.4.0	MEDIUM	1	Highest	33
json-smart-2.3.jar	cpe:2.3:a:json-smart_project:json-smart:2.3:*:*:*:*:*:*:* cpe:2.3:a:json-smart_project:json-smart-v2:2.3:*:*:*:*:*:*:*	pkg:maven/net.minidev/json-smart@2.3	HIGH	2	Highest	45
log4j-api-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1	HIGH	5	Highest	42
logback-core-1.2.3.jar	cpe:2.3:a:qos:logback:1.2.3:*:*:*:*:*:*:*	pkg:maven/ch.qos.logback/logback-core@1.2.3	HIGH	2	Highest	31
snakeyaml-1.25.jar	cpe:2.3:a:snakeyaml_project:snakeyaml:1.25:*:*:*:*:*:*:*	pkg:maven/org.yaml/snakeyaml@1.25	CRITICAL	8	Highest	44
spring-boot-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*:*:*	pkg:maven/org.springframework.boot/spring-boot@2.2.4.RELEASE	CRITICAL	8	Highest	39
spring-boot-starter-web-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*:*:* cpe:2.3:a:web_project:web:2.2.4:release:*:*:*:*:*:*:*	pkg:maven/org.springframework.boot/spring-boot-starter-web@2.2.4.RELEASE	CRITICAL	8	Highest	35
spring-core-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*:*	pkg:maven/org.springframework/spring-core@5.2.3.RELEASE	CRITICAL*	17	Highest	36
spring-hateoas-1.0.3.RELEASE.jar	cpe:2.3:a:vmware:spring_hateoas:1.0.3:release:*:*:*:*:*:*:*	pkg:maven/org.springframework.hateoas/spring-hateoas@1.0.3.RELEASE	MEDIUM	1	Highest	43
spring-web-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:web_project:web:5.2.3:release:*:*:*:*:*:*:*	pkg:maven/org.springframework/spring-web@5.2.3.RELEASE	CRITICAL*	18	Highest	34
spring-webmvc-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:web_project:web:5.2.3:release:*:*:*:*:*:*:*	pkg:maven/org.springframework/spring-webmvc@5.2.3.RELEASE	CRITICAL*	17	Highest	36
tomcat-embed-core-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.30	CRITICAL*	60	Highest	30
tomcat-embed-websocket-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-websocket@9.0.30	CRITICAL*	61	Highest	30

* indicates the dependency has a known exploited vulnerability

6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.



7. Summary

The refactoring of Artemis Financial's web application focused on addressing several important areas of software security identified during the vulnerability assessment process. These areas included secure communications, data integrity, authentication, and vulnerability management. Secure communications were established by converting the application from HTTP to HTTPS through the configuration of SSL/TLS in the application.properties file, using port 8443 and a self-signed certificate generated through Java Keytool. This enhancement ensures that sensitive client information is encrypted while traveling across the network and is protected from interception by unauthorized parties.

Data integrity was strengthened through the implementation of SHA-256 cryptographic hashing within the ServerController class. The application was refactored to generate a checksum that allows users to verify that transmitted data has not been altered or corrupted. A new /hash endpoint was added to return both the original data string and its SHA-256 checksum, providing a practical mechanism for validating file and message integrity.

Additional layers of security were implemented through the use of a self-signed certificate to authenticate the server and the execution of OWASP Dependency-Check to identify known

vulnerabilities within project dependencies. These changes support a defense-in-depth strategy by protecting data during transmission, verifying data integrity, authenticating communications, and reducing the risk of introducing vulnerable third-party components into the software environment. Together, these security enhancements significantly improve the application's ability to protect Artemis Financial's sensitive customer information and comply with secure software development practices.

8. Industry Standard Best Practices

Industry standard best practices were applied throughout the refactoring process to maintain and enhance the software application's existing security posture. The SHA-256 algorithm was selected because it is a NIST-approved hashing standard that remains widely trusted for integrity verification. HTTPS was implemented though TLS, which is the industry-standard protocol for securing web communications. Java Keytool was used to generate and manage certificates following established Public Key Infrastructure (PKI) practices, and OWASP Dependency-Check was used to identify known vulnerabilities in third-party libraries, aligning with OWASP security recommendations.

The refactoring process also followed the principle of defense in depth by implementing multiple layers of security rather than relying on a single protective mechanism. Secure communications through TLS protect data in transit, SHA-256 hashing protects data integrity, certificates provide authentication, and dependency scanning helps identify software supply chain risks. These overlapping controls reduce the likelihood that a single vulnerability could compromise the entire system.

Applying industry-standard best practices provides significant value to Artemis Financial's overall well-being. By implementing encryption, secure communications, cryptographic integrity verification, and ongoing vulnerability monitoring, the company can better protect sensitive financial information from unauthorized access and tampering. These practices help reduce the risk of costly data breaches, strengthen customer trust, support regulatory compliance efforts, and improve the long-term maintainability of the software. Following established secure coding practices also helps ensure that future enhancements can be developed on secure foundation while minimizing the introduction of technical debt and security vulnerabilities.

References:

- What is the SHA-256 Algorithm, and How Does It Work? (2026, February 10). https://www.theknowledgeacademy.com/blog/sha-256-algorithm/
- Anna. (2024, November 4). *Why PROTECTIMUS recommends the SHA256 algorithm - protectimus solutions*. Protectimus. https://www.protectimus.com/blog/why-protectimus-recommends-the-sha256-algorithm/